Date : __/__/____ Assignment - 7 Page No.: ____

Problem Statement - Implement stack as an ADT & apply it for diff. expression conversions.

Objective -

1) To study stack & its operations.
2) To study the importance of expression conversions.

Theory -

Stack - Special case of ordered list also called as restricted linear list where insertion & deletion happen at only one end called as top of stack. Elements are added to and removed from the top of the stack. The last element to be added is the first to be removed (LIFO). Only access to the stack is the top element.

Basic Stack Operations

- isEmpty()
- isFull()
- Push()
- Pop()

Application of Stack

- Expression Evaluation
- Infix to Postfix & Prefix conversion

- Function call management
- Recursion
- Parenthesis matching
- Undo/redo operations in editors
- String reversal

Expression conversion :

Expression conversion is the process converting an arithmetic expression from one notation to another using a sta

Three common notations are
- Infix
- Prefix
- Postfix

A stack is used to temporarily store ope & parentheses during the conversion.

Implementation :

Platform :
- 64-bit open source Linux or its derivatives
- Open source programming tools like gcc/ ~~Editored~~ Eclipse editor

Pseudocode :

```
If TOP == MAX - 1
    print "stack overflow"
else
```

```
    TOP = TOP + 1
    stack(TOP) = item
stop

POP():
if TOP == -1
    print "stack underflow"
else
    item = stack(TOP)
    TOP = TOP - 1
    return item
stop
```

Infix to Postfix

1) Initialize empty stack & postfix expression
2) Scan the infix expression from left to right.
3) If operand, add it to postfix
4) If '(', push it to postfix
5) If ')', pop & add operators until '(' is found.
6) After scanning, pop all remaining operators.
7) Display postfix expression.

Time Complexity

- Push - O(1)
- Pop - O(1)
- Infix to postfix - O(n)
- Space complexity - O(n)

Date : __/__/____

Page No.: ____

FAQs

1) What is the advantage of prefix & post over infix expression?

Prefix & postfix expressions do not require parentheses & operator precedence rules for evaluation, making them easier for computers to process.

2) Why is stack used in expression conversion

It is used to temporarily store operators & parentheses during expression conversion. It follows the LIFO principle.

3) Give stack full & stack empty cond. for

- Stack full : TOP == MAX - 1 → Overflow
- Stack empty : TOP == -1 → Underflow.

* Conclusion

Thus, implemented stack operations assignment using array concepts. This system is able to apply different expression conversion.

```c
#include <stdio.h>
#include <ctype.h>

#define MAX 100

char stack[MAX];
int top = -1;

void push(char ch)
{
    if (top == MAX - 1)
        printf("Stack Overflow\n");
    else
        stack[++top] = ch;
}

char pop()
{
    if (top == -1)
        return '\0';

    return stack[top--];
}

int precedence(char ch)
{
    if (ch == '^')
        return 3;
    if (ch == '*' || ch == '/')
        return 2;
    if (ch == '+' || ch == '-')
        return 1;
    return 0;
}
```

```
void infixToPostfix(char infix[])
{
    char postfix[MAX];
    int i = 0, j = 0;
    char ch;

    while (infix[i] != '\0')
    {
        ch = infix[i];

        if (isalnum((unsigned char)ch))
        {
            postfix[j++] = ch;
        }
        else if (ch == '(')
        {
            push(ch);
        }
        else if (ch == ')')
        {
            while (top != -1 && stack[top] != '(')
                postfix[j++] = pop();

            if (top != -1)
                pop();
        }
        else if (ch == '+' || ch == '-' ||
                 ch == '*' || ch == '/' || ch == '^')
        {
```

```
            while (top != -1 && stack[top] != '(' &&
                (precedence(stack[top]) >
precedence(ch) ||
                (precedence(stack[top]) ==
precedence(ch)
                && ch != '^')))
            {
                postfix[j++] = pop();
            }

            push(ch);
        }

        i++;
    }

    while (top != -1)
        postfix[j++] = pop();

    postfix[j] = '\0';

    printf("Postfix Expression: %s\n", postfix);
}

int main()
{
    char infix[MAX];

    printf("Enter an infix expression: ");
    scanf("%99s", infix);

    infixToPostfix(infix);

    return 0;
}
```